

Loading the battery dataset

▪ Battery Cell Data

The measurements from LIBs are collected during cycling tests under fast-charging conditions, using a 48-channel Arbin LBT potentiostat within a temperature-controlled chamber set at 30°C. This example focuses a single cell that is cycled to failure (80% state of health) in order to provide a comprehensive view of its performance over time. The cycling sequence exercises the battery cell with dynamic fast charging and constant 4C discharging. The cell data includes essential measurements and per-cycle summaries:

- Date Time: Recorded in date-time format, which may be reset during a cycle due to data collection errors.
- Cycle Index: An integer identifying the cycle number.
- Step Index: An integer identifying the steps within the fast-charging policy.
- Current: Measured in amperes (A).
- Voltage: Measured in volts (V).
- Temperature: Measured in degrees Celsius (°C).

```
dataFile
= matlab.internal.examples.downloadSupportFile("predmaint","batteryagingdata/
singlecell/v1/singleCellLifeTimeData.zip");
unzip(dataFile)
load("singleCellLifeTimeData.mat")
```

=== Single Cell Battery Life Time Dataset (First 5 Rows) ===

```
head(data,5)
```

Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index	Current	Voltage	Charge
1	0	13-May-20...	-1780	0	0	0	3.3018	
2	0.0001	13-May-20...	-1780	0	0	0	3.3018	
3	9.9983	13-May-20...	-1770	0	0	0	3.3018	
4	20.002	13-May-20...	-1760	0	0	0	3.3018	
5	30.001	13-May-20...	-1750	0	0	0	3.3018	

▪ Parsing and Segment data

To identify the cycling phases and modes and based on the measurements, use `batteryTestDataParser`. This function enables raw data visualization, aids in identifying anomalies, and prepare high-quality data.

```
bParser = batteryTestDataParser(data);
bParser.CurrentVariable = 'Current';
bParser.VoltageVariable= "Voltage";
bParser.TimeVariable = "DateTime";
bParser.CycleIndexVariable = 'Cycle_Index';
bParser.StepIndexVariable = 'Step_Index';
```

=== Single Cell Battery Life Time Dataset (First 5 Rows) - Updated ===

```
segmentedRawDataTable = segmentData(bParser);
head(segmentedRawDataTable,5)
```

Data_Point	Test_Time	DateTime	Step_Time	Step_Index	Cycle_Index	Current	Voltage	Charge
1	0	13-May-20...	-1780	0	0	0	3.3018	
2	0.0001	13-May-20...	-1780	0	0	0	3.3018	
3	9.9983	13-May-20...	-1770	0	0	0	3.3018	
4	20.002	13-May-20...	-1760	0	0	0	3.3018	
5	30.001	13-May-20...	-1750	0	0	0	3.3018	

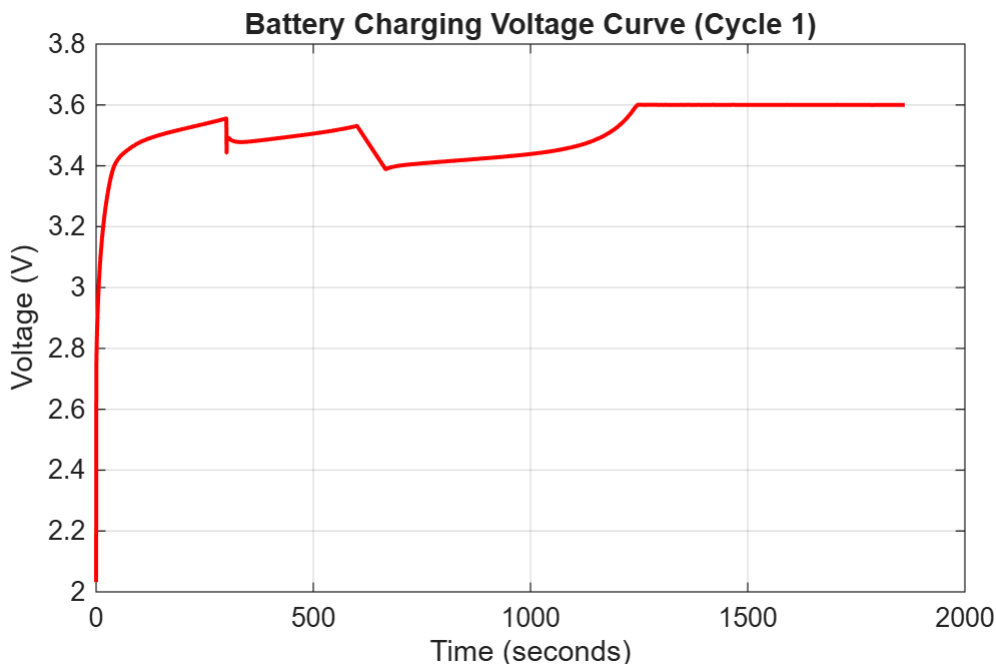
- Extract only the valid "Charge" phase data for Cycle 1

```
chargingData = segmentedRawDataTable(segmentedRawDataTable.Cycle_Index == 1 & ...
    segmentedRawDataTable.CyclingPhases == "Charge", :);
chargingData.TimeInSeconds = seconds(chargingData.DateTime -
chargingData.DateTime(1));
```

- Create the Plot (Voltage vs. Time)

```
figure;
plot(chargingData.TimeInSeconds, chargingData.Voltage, '-r', 'LineWidth', 1.5);

xlabel('Time (seconds)');
ylabel('Voltage (V)');
title('Battery Charging Voltage Curve (Cycle 1)');
grid on;
```



```
totalPoints = height(chargingData);
```

```

evenIndices = round(linspace(1, totalPoints, 20));

% 4. Extract those 20 specific points
evenTime = chargingData.TimeInSeconds(evenIndices);
evenVoltage = chargingData.Voltage(evenIndices);

% 5. Create the plot
figure;
% Plot a faint line in the background so you can see the trend,
% then overlay the 20 distinct points as red circles
plot(chargingData.TimeInSeconds, chargingData.Voltage, ':k', 'LineWidth', 1); %
Dotted background line
hold on;
plot(evenTime, evenVoltage, 'or', 'MarkerSize', 5, 'MarkerFaceColor', 'r'); % 20
Evenly-spaced points
hold off;

xlabel('Time (seconds)');
ylabel('Voltage (V)');
title('20 Battery Charging Voltage Points');
grid on;

```

